Email Draft and Analysis Note for Taylan G. Topcu

1. Email Draft (to Taylan G. Topcu)

Dear Taylan G. Topcu,

Thank you again for the thoughtful feedback and for taking the time to review the manuscript so carefully. Your comments are exactly the right ones for strengthening the framing for the systems engineering audience.

On scope: our generation input is the SysMBench natural-language requirements text (Box 1). The domain/grammar/difficulty labels (Boxes 4–6) are not prompt inputs; we use them for stratified evaluation. The generated artifact is textual SysML (Box 2-type output), and our loop is a syntactic compile/repair process rather than end-use validation.

Based on your suggestions, we ran two focused analyses:

1. SysMBench label-conditioned analysis (`fig3`, `fig6`, `fig9`, `fig10`)
2. SysIDE diagnostic analysis (`figE15`, `figE17`, `figE18`)

This gives us a tighter evidence chain for where errors concentrate, how quickly they repair, and what happens when they are not fixed immediately.

Headline findings:

- Label effects are visible, but support is imbalanced, so subgroup performance must be interpreted with sample size.
- Difficulty is not strongly monotonic in this campaign (weak linear fit).
- Error burden is highly concentrated: parsing and reference diagnostics dominate both single-shot and cumulative error counts.
- Persistence is mostly repeated exact diagnostics, not random family-level churn.

I agree with your terminology concern: "validator-driven" can be misread in SE language. We should shift to wording like "syntax-verifier-in-the-loop" and explicitly reserve "validation" for operational/end-user fitness.

Given page limits, my proposal is to keep a compact version of these results in the main paper and move expanded stratified/error-tail analysis to appendix (or journal version), with one concise pointer in the discussion.

If this direction looks good to you, I can integrate tracked edits directly.

Best regards,
Your Name

2. Technical Note: SysMBench Label Analysis + SysIDE Error Analysis

2.1 Data and Scope

- 151 unique prompts, evaluated across 4 models, for 604 model-prompt runs total.
- SysMBench labels used for stratified analysis: `domain`, `grammar` (key grammar label), `difficulty` (1–5).
- SysIDE iteration outputs used for error analysis: family-level compiler diagnostics across refine iterations.

2.2 SysMBench Metrics (Definitions)

`n_prompts` Number of prompts in a subgroup.

`first_shot_pass_rate` Percent passing on first generation attempt.

`eventual_pass_rate` Percent passing within iterative repair cap.

`unresolved_rate` Percent still failing at cap.

`first_failed_then_recovered_rate` Percent that fail first then pass later.

`mean_first_iteration_error_count` Average first-iteration error count.

`mean_total_error_count_across_iterations` Cumulative error count over loop iterations.

`mean_iterations_run` Mean iterations executed.

`mean_iterations_to_success` Mean iterations required to first pass.

`median_iterations_to_success` Median iterations required to first pass.

2.3 Label-Conditioned Findings (Figures 3, 6, 9, 10)

Figure 3 (Domain vs mean iterations-to-success). All-model domain means range from roughly 1.25 to 2.63 iterations. Higher means appear in low-support domains (e.g., Systems Engineering), while the dominant-volume domain (Vehicle Traffic) is lower at about 1.72. This supports reporting domain performance together with support.

Figure 6 (Top grammar labels). There are 47 total grammar labels; the top-15 labels cover 88/151 prompts (58.3%). Among those top-15, higher mean iterations occur for *Requirement* ($\sim$2.25), *Analysis and Trade* ($\sim$2.06), and *Language Extension* ($\sim$1.96).

Figure 9 (Difficulty vs mean iterations-to-success). Overall means are: difficulty 1 = 1.77, 2 = 1.67, 3 = 1.94, 4 = 1.67, 5 = 1.58. The linear trend fit is weak ($R^2 \approx 0.18$), indicating no strong monotonic relationship in this campaign.

Figure 10 (Support/sample size context). Label support is highly imbalanced: 13 domains, 47 grammar labels, and 5 difficulty levels, with difficulty levels 1–2 accounting for most runs. This figure is critical context for interpreting subgroup differences.

2.4 SysIDE Error Metrics (Definitions)

`single_shot_error_instances` Error family count at single-shot generation.

`total_error_instances` Error family count accumulated across all iterations.

`prompts_affected` Number of prompts where a family appears.

`episodes` Contiguous appearance blocks of an error family in one run.

`iterations_to_repair` Additional iterations until family clears.

`p_repair_le1`, `p_repair_le2` Repair probability within 1 or 2 additional iterations.

`long_tail_count` Episodes requiring 3 or more additional iterations.

`persistence_mode` One of: `same_exact_error_recurs`, `same_error_template_recurs`, `family_only_churn`.

2.5 Error Findings (Figures E15, E17, E18)

Figure E15 (Common errors: single-shot vs cumulative vs coverage). Parsing and reference errors dominate:

- `parsing-error`: 1144 single-shot, 1666 cumulative, 126 prompts affected.
- `reference-error`: 824 single-shot, 1224 cumulative, 92 prompts affected.

Top-2 families account for about 89% of single-shot errors and 89% of cumulative errors.

Figure E17 (Repair completion curves, top-5 families). Repair is fast for most episodes, but tails remain concentrated:

- `parsing-error`: 71.3% repaired within $\leq$1 additional iteration; 89.9% within $\leq$2.
- `reference-error`: 75.9% within $\leq$1; 93.8% within $\leq$2.
- `port-definition-owned-usages-not-composite`: 100% within $\leq$2.

Figure E18 (If not fixed immediately, what happens next?). Among persistent episodes:

- same exact diagnostic recurs: 84.8%
- same template recurs: 8.0%
- family-only churn: 7.2%

This indicates persistence is mostly repeated unresolved root-cause diagnostics rather than random type churn.

2.6 Messaging Implications for the Paper

1. Keep SE terminology precise: prefer "syntax-verifier-in-the-loop" over "validator-driven."
2. In the main paper, retain concise subgroup/error insights with support-aware framing.
3. Move expanded stratified and tail analyses to appendix or journal extension.